

Tests Applicatifs

du premier test vert au dépôt avec CI verte

*Travaux pratiques accompagnés, en Python et
TypeScript,
autour d'un mini gestionnaire de tâches.*

support remis aux étudiants par :

L'ÉQUIPE PÉDAGOGIQUE

PUBLIC : *Terminale NSI · BTS 2 · M1*

PRÉ-REQUIS : *Programmation, notions de Git*

LANGAGES : *Python (pytest) & TypeScript (Vitest)*

SORTIE ATTENDUE : *dépôt GitHub avec CI verte*

SOMMAIRE

Table des matières

AVANT-PROPOS	<i>Pourquoi ce travail pratique</i>	3
LEXIQUE	<i>Les huit notions à maîtriser</i>	4
PARTIE I	<i>Premier test vert (test unitaire, pattern AAA)</i>	5
PARTIE II	<i>Casser les fonctions (cas limites, entrées invalides)</i>	6
PARTIE III	<i>Faire dialoguer (intégration, fonctionnel)</i>	7
PARTIE IV	<i>Bug et régression (E2E, blindage)</i>	8
PARTIE V	<i>Livrer (intégration continue GitHub)</i>	9
ANNEXE A	<i>Mémo des commandes</i>	10
ANNEXE B	<i>Consignes de rendu</i>	11

« Un programme sans tests est un programme cassé qui n'a pas encore été observé. »

- adapté d'un adage en génie logiciel.

AVANT-PROPOS

Pourquoi ce travail pratique

Ce document accompagne un travail pratique en cinq parties progressives. Vous y construirez, à partir d'un dépôt GitHub fourni, une discipline solide de test logiciel autour d'une mini-application : un gestionnaire de tâches implémenté à la fois en *Python* et en *TypeScript*. L'idée directrice : un même problème, deux langages, les mêmes principes - ce qui rend transférable l'essentiel de ce que vous apprendrez ici.

Le parcours est conçu pour vous faire vivre *physiquement* ce qu'un test apporte. Vous écrirez d'abord un test qui passe au vert. Vous chercherez ensuite à *casser* votre propre code aux frontières, puis à le mettre en relation avec d'autres composants. Vous traquerez un bogue volontaire, vous écrirez le test qui le démasque, et vous garderez ce test à vie pour empêcher son retour. Enfin, vous publierez votre travail sur GitHub avec une chaîne d'intégration continue qui valide automatiquement chacun de vos commits.

1 Compétences ciblées

1. Écrire un test unitaire propre, en pattern *arrange-act-assert*.
2. Identifier les cas limites et les entrées invalides d'une fonction.
3. Distinguer test unitaire, test d'intégration, test fonctionnel et test *end-to-end*.
4. Reproduire un bogue par un test, le corriger, conserver le test en garde-fou de régression.
5. Pousser un dépôt sur GitHub avec une chaîne d'intégration continue verte.
6. Adopter une discipline de commits lisible (commits conventionnels).

2 Conventions du document

DÉFINITION 2.1

Boîte de définition. Lorsque cette boîte apparaît, elle pose le sens précis d'un mot du métier (test unitaire, fixture, régression, etc.).

EXEMPLE 2.2

Boîte d'exemple. Lorsque cette boîte apparaît, elle illustre une notion par un cas concret tiré du code du projet.

EXERCICE 2.3

Boîte d'exercice. Lorsque cette boîte apparaît, c'est à vous de coder. Le travail est obligatoire ; il sera évalué par votre formateur.

REMARQUE 2.4

Boîte de remarque. Elle apporte un complément, une astuce ou une mise en garde non bloquante.

3 Projet support

Vous travaillerez sur un *gestionnaire de tâches*. Une tâche possède un titre, une description, une priorité (`low`, `medium`, `high`), un statut (`todo`, `doing`, `done`) et une éventuelle date d'échéance. Les tâches sont stockées dans un fichier *JSON*. L'application est livrée en deux exemplaires : l'un en Python, l'autre en TypeScript. *Vous pouvez travailler dans le langage de votre choix*, ou faire les deux en parallèle pour mieux mémoriser les principes.

LEXIQUE

Les huit notions à maîtriser

Le métier du test est très bavard. Voici la liste minimale du vocabulaire qui reviendra dans tout le document.

TEST UNITAIRE L.1

Test qui vérifie une *seule unité* de comportement (typiquement une fonction), isolée du reste du système.

CAS LIMITES L.2

Tests aux frontières du domaine de définition : longueur 0, longueur maximale, longueur maximale plus un, liste vide, valeur nulle, valeur maximale, etc.

ENTRÉES INVALIDES L.3

Tests qui vérifient que le code *refuse* proprement (par une exception explicite) les valeurs qui ne respectent pas le contrat de la fonction.

TEST D'INTÉGRATION L.4

Test qui vérifie que *plusieurs modules* coopèrent correctement (par exemple : la couche métier et la couche de persistance).

TEST FONCTIONNEL L.5

Test orienté *règle métier*, exprimé du point de vue de l'utilisateur final. On ne se soucie pas de l'implémentation, seulement du résultat observable.

TEST BOUT EN BOUT (E2E) L.6

Test qui simule un *parcours utilisateur complet*, de bout en bout, par les mêmes points d'entrée que l'application réelle.

TEST DE RÉGRESSION L.7

Test gardien d'un bogue déjà rencontré : s'il échoue de nouveau, c'est que le bogue est *revenu*. Il sert de filet de sécurité permanent.

TEST DE VALIDATION L.8

Vérification finale que le produit livré *satisfait l'ensemble du cahier des charges*.

L.9 La pyramide des tests

E2E : peu, lents, précieux

Intégration : ciblés, modérés

Unitaires : nombreux, rapides, isolés

Figure 1 - heuristique de répartition : 70 % unitaires, 20 % intégration, 10 % E2E.

Cette pyramide n'est pas une loi ; c'est une *boussole*. Lorsqu'un projet inverse la pyramide (beaucoup de tests E2E, peu de tests unitaires), il devient lent à valider et coûteux à maintenir. À votre niveau, retenez surtout : *écrivez beaucoup de tests unitaires*, complétez par quelques tests d'intégration, puis quelques tests bout-en-bout pour les parcours critiques.

PARTIE I

I. Premier test vert

Objectif : écrire un test unitaire propre en pattern arrange-act-assert, et l'observer passer au vert pour la première fois.

I.1 Le pattern AAA

Tout test bien écrit suit trois étapes : on *prépare* le contexte, on *déclenche* le comportement, puis on *affirme* le résultat attendu. Cette discipline rend les tests immédiatement lisibles par un·e collègue qui ne connaît pas votre code.

<i>A</i> ARRANGE <i>on prépare la scène, les variables, l'objet à tester.</i>	<i>A</i> ACT <i>on déclenche le comportement à observer.</i>	<i>A</i> ASSERT <i>on vérifie que le résultat est celui qu'on attend.</i>
---	--	---

Figure 2 - les trois temps d'un test bien construit.

I.2 Un même test, deux langages

PYTHON · PYTEST

```
# Arrange / Act / Assert
def test_task_se_cree_avec_titre():
    titre = "Faire les courses"
    tache = Task(id=1, title=titre)
    assert tache.title == titre
```

TYPESCRIPT · VITEST

```
// Arrange / Act / Assert
it("se cree avec un titre", () => {
    const titre = "Faire les courses";
    const t = new Task({ id: 1, title: titre });
    expect(t.title).toBe(titre);
});
```

I.3 Travail à réaliser

EXERCICE 1.3.1 - PREMIERS TESTS VERTS

1. Ouvrez le fichier `python/tests/test_unitaire_task.py` (ou son équivalent TypeScript).
2. Exécutez la commande `pytest` dans le dossier `python/` et observez la liste de tests verts. Faites de même côté TypeScript avec `npm test`.
3. Repérez les zones marquées `TODO ÉLÈVE` en bas du fichier.
4. Ajoutez *au moins trois* tests unitaires de votre choix. Suggestions : description optionnelle, format de date ISO, comparaison par `to_dict()`.
5. Effectuez un commit : `git commit -m "test(unitaire): ajoute tests sur Task"`

REMARQUE 1.3.2 - RÈGLE D'OR

Un test qui n'a *jamais* échoué n'a *jamais* rien prouvé. Si vous doutez d'un test, cassez volontairement le code l'espace d'un instant pour le voir passer au rouge, puis remettez tout en ordre. Cette vérification est un réflexe à acquérir.

PARTIE II

II. Casser les fonctions

Objectif : anticiper ce que d'autres feront dire à votre code. Tester aux frontières, et tester les entrées qu'il doit refuser.

II.1 Cas limites

On appelle *cas limite* toute valeur située à la frontière du domaine de définition d'une fonction. C'est là, statistiquement, que se cachent la majorité des bogues réels.

EXEMPLE II.1.1 - UN TITRE DE LONGUEUR MAXIMALE

Si la spécification dit « le titre fait au plus 100 caractères », alors : 99 doit passer, 100 doit passer, 101 doit échouer. On teste les trois - pas seulement « un titre normal ».

II.1.2 Liste des frontières à éprouver

- Chaînes : longueur 0, 1, max, max+1, chaîne d'espaces uniquement.
- Listes : vide, un élément, gros volume (1000+ éléments).
- Nombres : 0, -1, valeur maximale, division par zéro.
- Dates : 29 février d'une année non bissextile, format ISO ou autre.

II.2 Entrées invalides

Une entrée invalide n'est pas une frontière, c'est un refus. Le code doit *lever une exception explicite* plutôt que de planter silencieusement ou de produire un résultat incorrect.

PYTHON • PYTEST

```
def test_titre_vide_rejete():
    with pytest.raises(InvalidInputError):
        validate_title("")
```

TYPESCRIPT • VITEST

```
it("titre vide est rejete", () => {
    expect(() =>
        validateTitle("")
    ).toThrow(InvalidInputError);
});
```

EXERCICE II.3 - FRONTIÈRES ET REJETS

1. Ouvrez `test_cas_limites.py` et `test_entrees_invalides.py` (et leurs équivalents TS).
2. Lancez les tests, observez les verts existants.
3. Ajoutez *au moins deux* tests de cas limites et *au moins un* test d'entrée invalide.
4. Pour chacun, justifiez par un commentaire la frontière ou la règle métier visée.
5. Commit : `git commit -m "test(cas-limites): ajoute frontiere max sur titre"`

ASTUCE 11.4 - LA TECHNIQUE DU « ET SI... »

Pour chaque paramètre d'une fonction, demandez-vous : *et si c'était vide ? et si c'était énorme ? et si c'était négatif ? et si c'était d'un autre type ?* Chaque « et si » qui devrait provoquer un refus correspond à un test à écrire.

PARTIE III

III. Faire dialoguer les modules

Objectif : passer du grain fin à la vue d'ensemble. Vérifier que plusieurs morceaux coopèrent correctement, et qu'un parcours métier reste cohérent.

III.1 Intégration vs fonctionnel

TEST D'INTÉGRATION	TEST FONCTIONNEL
Vérifie que <i>deux modules techniques</i> se parlent bien. Ex. : <code>task_manager</code> et <code>storage</code> coopèrent pour sauvegarder puis recharger.	Vérifie une <i>règle métier</i> du point de vue de l'utilisateur. Ex. : « quand je marque une tâche terminée, les statistiques le reflètent ».
Concerne souvent le disque, la base, le réseau.	Concerne les valeurs, les comportements visibles.

III.2 Patron : un aller-retour disque

```
# 1. On cree des taches en memoire
mgr = TaskManager()
mgr.create_task("Tache A", priority="high")

# 2. On sauve sur disque (fichier temporaire isole)
save_tasks(str(tmp_path / "taches.json"), mgr.list_tasks())

# 3. On recharge dans un manager neuf
mgr2 = TaskManager()
mgr2.replace_all(load_tasks(str(tmp_path / "taches.json")))

# 4. On verifie que tout est revenu
assert mgr2.list_tasks()[0].title == "Tache A"
```

REMARQUE III.2.1 - ISOLATION DES TESTS

En pytest, la *fixture* `tmp_path` fournit un dossier temporaire propre, automatiquement supprimé après le test. En Vitest, on utilise `mkdtempSync`. Ne partagez *jamais* un fichier de test entre plusieurs tests : c'est la garantie de tests instables (*flaky*).

EXERCICE III.3 - DIALOGUER ET RACONTER

1. Ouvrez `test_integration.py` et `test_fonctionnel.py`.
2. Lisez les tests existants, observez l'usage de `tmp_path`.
3. Ajoutez *un* test d'intégration qui modifie une tâche puis vérifie la persistance.
4. Ajoutez *un* test fonctionnel sur la règle métier de votre choix (par exemple : « le filtre `status="done"` ne renvoie que des tâches terminées »).

5. Commit: `git commit -m "test(integration): aller-retour save/load"`

PARTIE IV

IV. Un bogue, et son blindage

Objectif : vivre le rituel complet du bogue. Le démasquer par un test rouge, le corriger jusqu'au vert, et le sceller dans un test de régression qui empêchera son retour.

IV.1 Le rituel du bogue

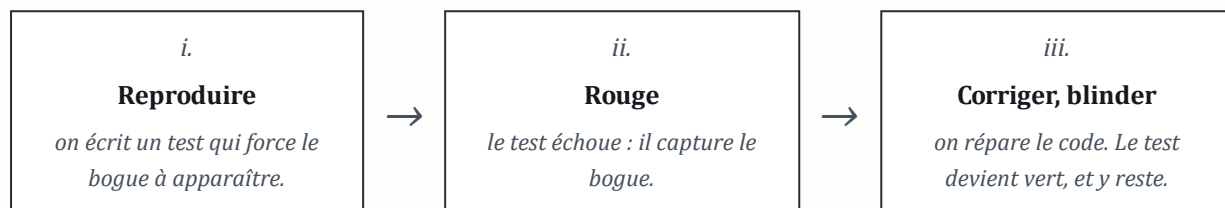


Figure 3 - trois temps successifs pour traiter un bogue en méthode test-first.

IV.2 Votre bogue, n° 001

Dans le module `task_manager.py` (et son équivalent TypeScript), la méthode `delete_task` ne fait *pas* ce que son nom prétend. Le scénario suivant le révèle : on crée trois tâches d'identifiants 1, 2, 3 ; on appelle `delete_task(2)` ; on examine ce qu'il reste. Le résultat attendu est `{1, 3}`. Le résultat observé est différent.

EXERCICE IV.3 - DÉMASQUER, RÉPARER, BLINDER

1. Ouvrez `test_regression.py`. Lisez la marche à suivre en tête de fichier.
2. Écrivez un test qui *doit échouer* en l'état actuel du code. Lancez la suite : confirmez le rouge.
3. Ouvrez `task_manager.py`. Lisez attentivement la méthode `delete_task`. Identifiez la confusion entre *identifiant* et *indice de liste*.
4. Corrigez le code. Relancez la suite : tout doit redevenir vert.
5. Ajoutez un *second* test de régression couvrant une variante du même bogue (par exemple, suppression d'une tâche inexistante).
6. Effectuez un commit explicite : `git commit -m "fix(bug-001): delete_task supprime par id"`

REMARQUE IV.4 - POURQUOI CONSERVER LE TEST

Dans six mois, un·e collègue réécrira `delete_task` pour une raison ou une autre. Si elle ou il réintroduit la confusion, votre test passera au rouge : c'est votre garde-fou automatique, transmissible.

IV.5 Et l'E2E, dans tout cela ?

Une fois le bogue corrigé, ouvrez `test_e2e.py`. Vous y trouverez un scénario complet de « journée de travail » : création de trois tâches, marquage, sauvegarde, fermeture, relance. Ce test ne teste pas *une* méthode, mais le *parcours utilisateur* dans son ensemble. Lisez-le, vérifiez son passage au vert, puis ajoutez un second scénario E2E de votre choix.

PARTIE V

V. Livrer : intégration continue

Objectif : votre travail sort de votre machine. Il est désormais validé automatiquement, à chaque push, par un robot d'intégration continue. Lorsqu'il devient vert, vous arborez votre premier badge professionnel.

V.1 Le fichier de chaîne d'intégration

Le dépôt fourni contient déjà un fichier `.github/workflows/ci.yml`. À chaque fois que vous poussez du code sur GitHub, deux tâches s'enchaînent en parallèle, l'une pour Python, l'autre pour TypeScript.

```
# Extrait - .github/workflows/ci.yml
jobs:
  python-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
      - run: pip install -r requirements.txt
      - run: pytest
  typescript-tests:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/setup-node@v4
      - run: npm install
      - run: npm test
```

V.2 Le badge à coller dans votre README

```
! [CI] (https://github.com/<votre-pseudo>/tp-tests-applicatifs/actions/workflows/ci.y
```

EXERCICE V.3 - PASSER EN VERT, ET LE MONTRER

1. Créez votre propre dépôt GitHub `tp-tests-applicatifs` (public ou privé ; partagez le lien avec votre formateur).
2. Poussez l'intégralité de votre travail. Ouvrez l'onglet *Actions* sur GitHub et observez l'exécution de la chaîne d'intégration.
3. Si la chaîne est rouge, lisez les journaux d'exécution, corrigez, puis poussez de nouveau. C'est le rituel.
4. Une fois la chaîne verte : ajoutez le badge en haut du `README.md`.
5. Ajoutez en bas du README une section « *Ce que j'ai appris* » de cinq lignes.

V.4 Validation finale

Vérifiez, avant remise, chacun des points suivants :

- ☐ Les tests passent tous en local (`pytest` et `npm test`).
- ☐ Le bogue n° 001 est corrigé ; ses tests de régression sont en vert.
- ☐ Chaque `TODO ÉLÈVE` a été remplacé par au moins un test ajouté.
- ☐ La chaîne d'intégration continue est verte sur GitHub.
- ☐ Le badge est visible dans le README.
- ☐ La section « *Ce que j'ai appris* » est rédigée.
- ☐ Au moins dix commits lisibles, en convention `type(scope) : message`.

ANNEXE A

Mémo des commandes

A.1 pytest - l'essentiel

```
pytest                # exécute toute la suite
pytest -k titre        # tests dont le nom contient "titre"
pytest -m unitaire     # filtre par marker
pytest -m "not e2e"    # exclut un marker
pytest --tb=short      # traceback courte
pytest --cov=src       # rapport de couverture
pytest -x              # s'arrête au premier échec
```

A.2 vitest - l'essentiel

```
npm test              # exécute toute la suite
npm run test:watch    # mode watch
npm run test:coverage # rapport de couverture
npx vitest run -t "casLimites" # filtre par nom
```

A.3 Assertions de référence

PYTEST

```
assert x == y
assert x is None
assert 'a' in liste
with pytest.raises(Err): ...
tmp_path
@pytest.fixture
@pytest.mark.unitaire
```

VITEST

```
expect(x).toBe(y)
expect(x).toEqual(obj)
expect(x).toBeNull()
expect(arr).toContain('a')
expect(() => fn()).toThrow(Err)
beforeEach / afterEach
describe / it
```

A.4 Git - survie

```
git status            # où en suis-je ?
git add <fichier>     # préparer un fichier
git commit -m "test(...): ..." # graver dans l'historique
git push -u origin <branche> # envoyer sur GitHub
git checkout -b partie-X-nom # nouvelle branche
git log --oneline -10  # dix derniers commits
```

A.5 Commits conventionnels

TYPE	QUAND L'UTILISER
feat	nouvelle fonctionnalité
fix	correction d'un bogue

TYPE	QUAND L'UTILISER
test	ajout ou modification de tests
refactor	réécriture sans changement de comportement
docs	documentation, README
chore	maintenance, dépendances, CI

ANNEXE B

Consignes de rendu

B.1 Ce que vous remettez

L'unique livrable attendu est *le lien vers votre dépôt GitHub*. Ce dépôt doit être accessible à votre formateur - public, ou privé avec lui ou elle en collaborateur.

B.2 Critères de validation

- Lecture du `README.md` claire, badge d'intégration continue visible.
- Chaîne d'intégration continue *verte* sur la branche principale au moment du rendu.
- Chaque `TODO ÉLÈVE` remplacé par au moins un test pertinent.
- Bogue n° 001 corrigé ; tests de régression présents et verts.
- Historique Git lisible : au minimum dix commits clairs, en convention.

B.3 Ce qui sera sanctionné

À ÉVITER

- Supprimer ou commenter un test pour faire passer la chaîne en vert.
- Désactiver les tests de régression du bogue n° 001.
- Messages de commit opaques (`"update"`, `"fix"`, `"final"`).
- Copie de l'historique Git d'un·e camarade : votre historique sera examiné.

B.4 Pour aller plus loin

- Couverture supérieure à 85 % (`pytest --cov=src` et `npm run test:coverage`).
- Une *pull request* par partie, plutôt qu'un *push* direct sur `main`.
- Un fichier `CHANGELOG.md` retraçant votre parcours.
- Une chaîne d'intégration qui publie le rapport de couverture en artefact.

« La meilleure fonction que vous écrierez dans votre carrière sera peut-être un test qui éclatera l'année prochaine et qui vous sauvera la mise. »

- le vous de demain, reconnaissant.